OrCa Showcase on `lido/core` — Kickoff

Capability check focused on properties **OrCa/`[V]` express that Foundry cannot** (or cannot without significant ghost-state machinery). Standard storage invariants — solvency, monotonicity, owner sync — are table stakes for any invariant fuzzer; they're not the point of this run.

Target

`WithdrawalQueueERC721` + `Lido` (real, no mocks). `lido.handleOracleReport` invoked with `sender` pinned to AccountingOracle address from locator; real `_processRewardsAndWithdrawals → wq.finalize` path executes. Actors: `user1`, `user2`, `oracle_addr`, `attacker`.

Three pillars

1. Liveness under fairness

Foundry's invariant mode is bounded reachability — there is no semantic notion of "eventually". `[V]` has both LTL operators and a first-class `fair:` section that OrCa uses for special reasoning, not just as syntactic sugar.

```
fair: [] <> finished(lido.handleOracleReport)

spec: [] (finished(wq.requestWithdrawals)
          ==> <> finished(wq.claimWithdrawal))
```

Reads: under a fair oracle, every request is eventually claimable. Without `fair`, the property trivially fails (oracle never reports). With `fair`, OrCa hunts for any counterexample where claim is permanently blocked despite reports arriving.

Foundry equivalent: doesn't exist. Closest approximation is bounded `assertTrue(claim_reached_within_N_steps)` — different property, different guarantee.

2. Inter-transaction coupling via sequence operator

`[V]`'s `;` lets two transactions be linked with argument-level coupling in one line. Foundry needs handler ghost state to express the same.

```
spec: [] (finished(wq.transferFrom(_, to, reqId)) ;
          finished(wq.claimWithdrawal(reqId))
          ==> eth_received_by(to))
```

Reads: if NFT for `reqId` is transferred to `to`, then the next claim of that `reqId` must pay `to` (not the original requester).

This is the bug class where `WithdrawalRequest.owner` (separate storage) and ERC721 ownership drift apart — and where claim could pay the wrong address. The property is stateful (tracks `reqId`-to-recipient), multi-actor (transfer and claim from different senders), and crosses two storage representations.

Foundry equivalent: handler with `mapping(uint256 => address) ghost_lastTransferTo`, post-claim assertion. Works, but requires writing and maintaining the handler — and the property is buried in handler logic instead of being a one-line spec.

3. State implications without enumeration

`[V]` free variables let the SMT solver hunt for a violating valuation directly. The spec implies a global state property (locked ether positive) from a per-request condition (any unclaimed-finalized request exists) — solver finds the witness, no manual scan.

```
vars: uint256 reqId

inv: (reqId > 0 && reqId <= wq.getLastFinalizedRequestId() && !wq.queue[reqId].cl
     ==> wq.getLockedEtherAmount() > 0
```

Reads: if any finalized-but-unclaimed request exists, locked ether must be positive.

This is solvency-adjacent and crosses per-request state with a global accounting variable. Violation would mean either lockedEther was decremented incorrectly during claim, or finalize raised the frontier without locking ether — both real bug classes.

Foundry equivalent: scan the queue every check to determine whether any claimable request exists, then assert. Gas-bounded queue depth in tests, and the assertion is structurally awkward — existence-check followed by global property, in a language designed for unit tests.

Hints

```
hints:
    finished(lido.submit(referral),
        sender := elem_in_list([user1, user2]);
```

```
    value  := rand_int(100, 100000000000000000000)
);

finished(wq.requestWithdrawals(amounts, owner),
    sender  := elem_in_list([user1, user2]);
    owner   := sender;
    amounts := [rand_int(100, 100000000000000000000)]
);

finished(lido.handleOracleReport(...),
    sender                        := oracle_addr;
    timeElapsed                   := 86400;
    clValidators                  := rand_int(1, 1000);
    clBalance                     := rand_int(0, 320000000000000000000000);
    withdrawalVaultBalance        := 0;
    elRewardsVaultBalance         := 0;
    sharesRequestedToBurn         := 0;
    withdrawalFinalizationBatches := [rand_int(1, 100)];
    simulatedShareRate            := <see open question>
);

finished(wq.claimWithdrawal(requestId),
    sender := elem_in_list([user1, user2])
);

finished(wq.transferFrom(from, to, tokenId),
    sender := from;
    from   := elem_in_list([user1, user2]);
    to     := elem_in_list([user1, user2])
)
```

Success criteria (one per pillar)

1. **Liveness:** spec holds with `fair`; falsified without it. Fairness reasoning is load-bearing, not decorative.
2. **Sequence:** spec holds, and at least one trace exercises `transferFrom ; claimWithdrawal` from different senders with recipient checked correctly.
3. **State implication:** spec holds across traces where finalized-but-unclaimed requests coexist with claimed ones.

Open questions

1. `simulatedShareRate` must equal what Lido computes from `clBalance` and other report

fields, otherwise `handleOracleReport` reverts on the sanity check and finalize coverage degrades to ~0 silently. Need either a `[V] solve` block tying it to other report fields, or a small helper that computes it before each report. **Single blocker for the run producing meaningful coverage.**

2. Free-variable semantics in `inv:` — verify violation-search behaves as universal quantification (OrCa hunts for any `reqId` that breaks the implication). Cross-check with Veridise before kickoff.